

Presentación Ejecutiva: Ecommify

Arquitectura de Datos Híbrida

Diapositiva 1: Contexto y Reto

- **¿Qué hicimos?** Diseñar un modelo de datos robusto para la plataforma Ecommify utilizando el dataset de Olist.
 - **¿Por qué lo hicimos?** Los e-commerce modernos exigen transacciones seguras (pagos/pedidos) al mismo tiempo que requieren alta disponibilidad para búsquedas masivas. Un único motor de base de datos se convierte en un cuello de botella.
 - **Solución Propuesta:** Arquitectura Híbrida (PostgreSQL + MongoDB).
-

Diapositiva 2: PostgreSQL - El Núcleo Transaccional

- **Cómo lo diseñamos:** Esquema relacional normalizado hasta 3FN para entidades core (clientes, pedidos, pagos, ítems).
 - **Por qué PostgreSQL:** Garantiza cumplimiento ACID. Nadie pierde dinero ni inventario en un pico de tráfico.
 - **Innovación:** Implementación de particionamiento por rangos en la tabla `orders` para archivar pedidos antiguos sin mermar la velocidad de los pedidos actuales.
-

Diapositiva 3: Potenciando PostgreSQL (Tipos Avanzados y Extensiones)

- **¿Cómo se hizo?** Se usó JSONB para especificaciones variables y Arrays (TEXT[]) para múltiples fotografías.
 - **El porqué de las extensiones:**
 - `PostGIS` : Calculamos logística de envíos basados en radio de cercanía real (geometría espacial), no solo coincidencia de strings.
 - `pg_trgm` : Tolerancia a fallos tipográficos. Si el usuario escribe "smartphne", la base de datos lo encuentra rápido gracias a un índice GIN sobre trigramas.
-

Diapositiva 4: MongoDB - Catálogo y Analítica

- **¿Qué hicimos?** Descargar las consultas de sólo lectura del motor relacional hacia un motor documental.
 - **¿Por qué lo hicimos?** Embeber las reseñas (reviews) dentro de la colección del producto evita hacer complejos y lentos JOINS cada vez que un usuario abre la página de un producto. Alta disponibilidad y lectura rápida.
-

Diapositiva 5: Análisis del Teorema CAP

- **El equilibrio arquitectónico (Por qué funciona):**
 - *Módulo Financiero/Pedidos (PostgreSQL):* Prioriza la **Consistencia (CP)**. Los datos deben ser 100% exactos al cobrar.

- *Módulo Catálogo/Logs (MongoDB)*: Prioriza la **Disponibilidad (AP)**. Ante una falla de red masiva, los clientes aún podrán navegar y ver productos, salvando el Customer Experience.
-

Diapositiva 6: Plan de Implementación

- **Cómo lo llevamos a producción:**
 1. Despliegue del esquema particionado de PostgreSQL.
 2. Migración de datos limpios y uso de `ST_SetSRID` para puntos geográficos.
 3. Implementación de MongoDB para frontend.
 4. Sincronización eventual de stock (PostgreSQL) hacia el catálogo (MongoDB) vía colas/CDC.